



3. Networking

Celso Paím



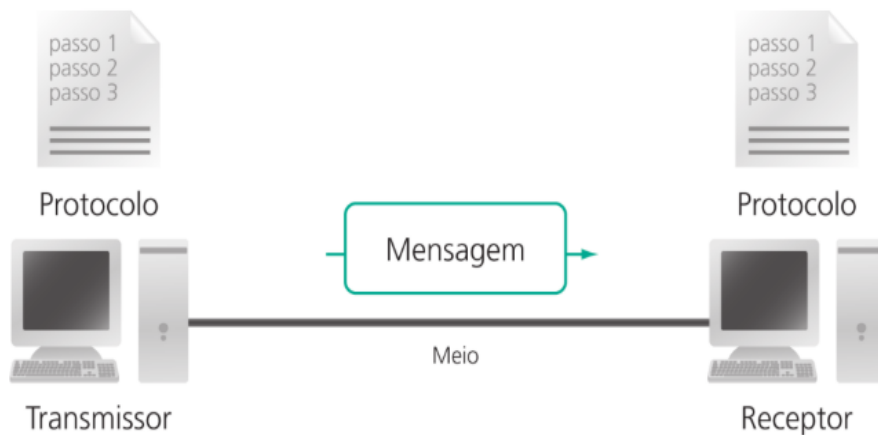
Universidade Católica de Angola

Faculdade de Engenharia Informática

Sistemas Distribuídos e Paralelos I

3. Networking

- O Java oferece uma API de rede abrangente (`java.net`) que permite aos programadores criarem aplicações robustas, escaláveis e habilitadas para funcionarem em rede.
- Para escrever programas que funcionem em rede (camada de aplicação) é necessário que conheçamos como funciona a comunicação entre computadores (esquema de comunicação, **modelos como OSI ou TCP/IP**, **protocolos**, **endereçamentos**, etc...)



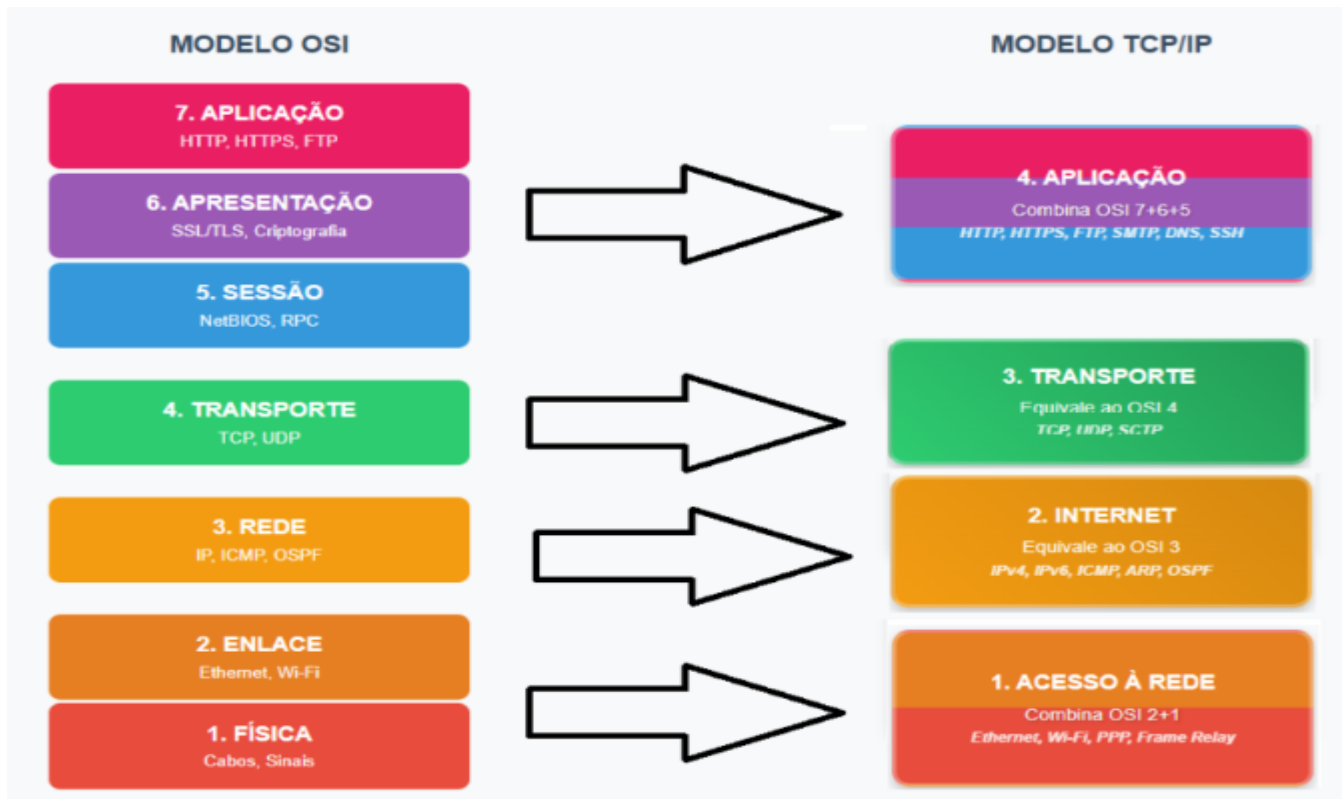


Universidade Católica de Angola

Faculdade de Engenharia Informática

Sistemas Distribuídos e Paralelos I

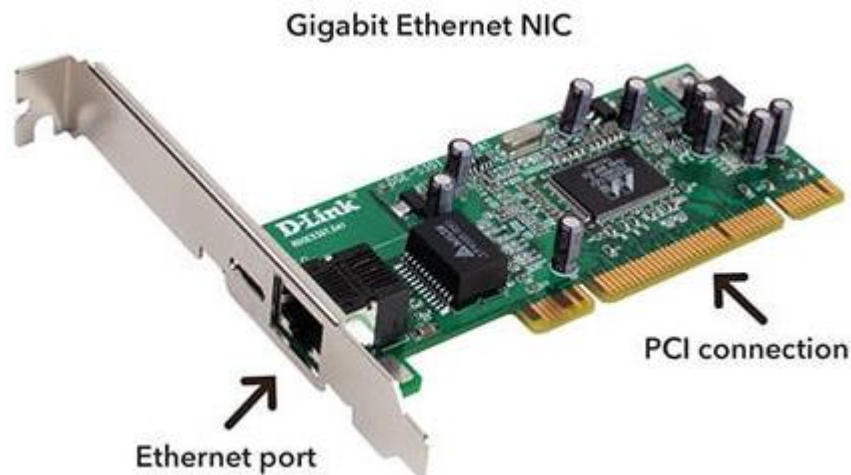
3. Networking (modelos e protocolos)



Um **protocolo de rede** é um conjunto padronizado de regras, convenções e procedimentos que governa a comunicação, formatação e transferência de dados entre dispositivos em uma rede.

3. Networking (endereçoçamento)

- **Endereço MAC:** é um identificador físico exclusivo e permanente, atribuído pelo fabricante a cada placa de rede de um dispositivo (Wi-Fi, Ethernet ou Bluetooth).
- Tem 48 bits e geralmente expresso em hexadecimal
- Usado na camada de Enlace





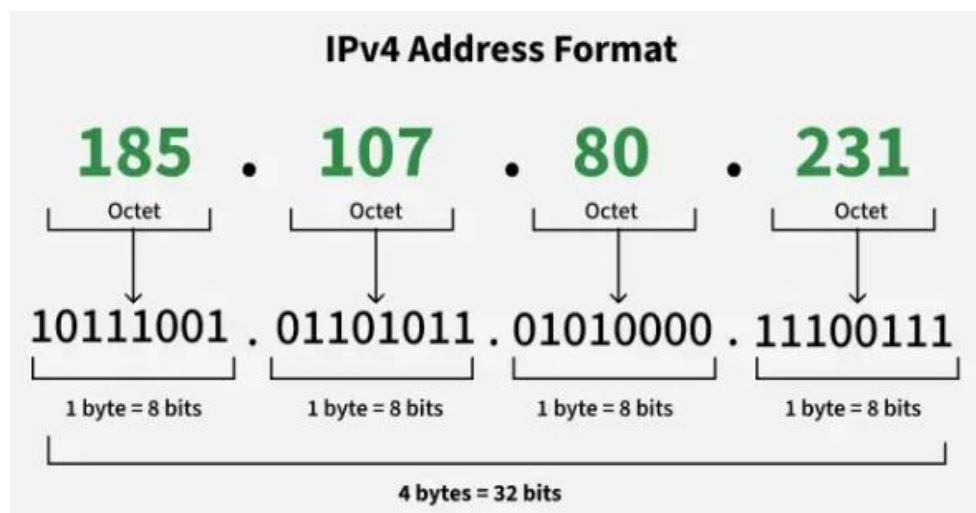
Universidade Católica de Angola

Faculdade de Engenharia Informática

Sistemas Distribuídos e Paralelos I

3. Networking (endereçoamento)

- **Endereço IP:** é o número de identificação atribuído a cada dispositivo conectado à uma rede.
- Os endereços IP têm duas versões ou padrões distintos. O IPv4 com 32 bits (até 4 bilhões de endereços IP), geralmente expresso como 4 octectos decimais, e o IPv6 (trilhões de endereços IP), expresso em hexadecimal.
- Os IP podem ser públicos ou privados e atribuídos de forma estática e dinâmica.
- Usado na camada de Rede
- Conceitos de *unicast*, *multicast*, *broadcast* e *anycast*.





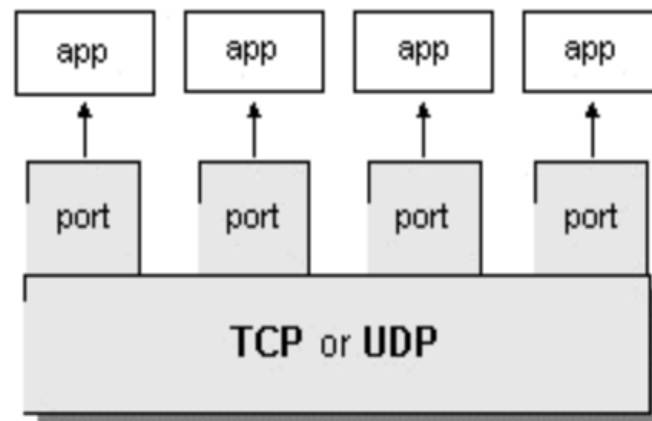
Universidade Católica de Angola

Faculdade de Engenharia Informática

Sistemas Distribuídos e Paralelos I

3. Networking (endereçamento)

- **Endereço porto (porta)** Portos TCP e UDP são endpoints numerados (de 0 a 65.535) usados por aplicações para enviar e receber dados em uma rede, funcionando como "portas de entrada/saída" específicas.
- Portos podem ser bem conhecidos (0 a 1.023), registados (1.024 a 49.151) ou não registados/dinâmicos/privados (49.152 a 65.535)
- O TCP é confiável, orientados a conexão e garante a entrega, enquanto o UDP é rápido, sem conexão e não garante a entrega
- Usados na camada de Transporte
- Ao conjunto IP:port é chamado de **socket**.





Universidade Católica de Angola

Faculdade de Engenharia Informática

Sistemas Distribuídos e Paralelos I

3.1. Java Networking

- A infraestrutura para rede Java é construída sobre o pacote `java.net`, que disponibiliza classes e interfaces para lidar com:
 - Endereços IP e nomes de host
 - Sockets para comunicação
 - Conexões URL para acessar recursos da web
 - Datagramas para comunicação UDP



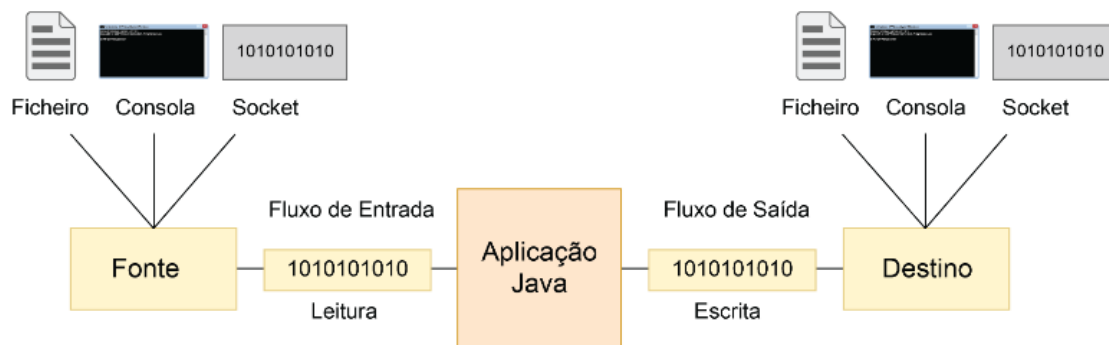
Universidade Católica de Angola

Faculdade de Engenharia Informática

Sistemas Distribuídos e Paralelos I

3.2. Classes em `java.net` e `java.io`

- Por meio das classes em `java.net`, os programas Java podem usar TCP ou UDP para se comunicar através de uma rede.
- As classes `URL`, `URLConnection`, `Socket` e `ServerSocket` usam TCP para comunicação em rede. Já as classes `DatagramPacket`, `DatagramSocket` e `MulticastSocket` são utilizadas com UDP.
- Em Java um *stream* (fluxo) é uma sequência de *bits* e *bytes* organizados de forma a comporem dados que podem ser usados para entrada ou saída (pacotes `java.io`, `java.nio` e `nio2`). No topo da hierarquia do pacote `java.io` estão as classes abstractas `OutputStream` e `InputStream` (e outras para representar ficheiros).





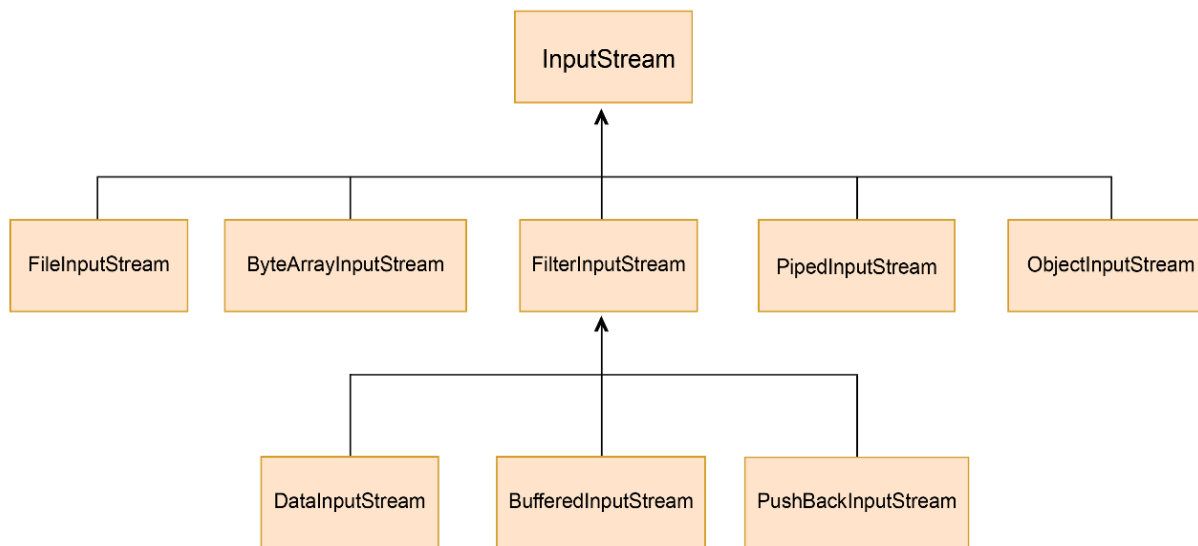
Universidade Católica de Angola

Faculdade de Engenharia Informática

Sistemas Distribuídos e Paralelos I

3.2.1 Classes em `java.io`

- `InputStream` é uma classe abstracta que oferece a funcionalidade básica para a leitura de um *byte* ou de uma sequência de *bytes* a partir de alguma fonte





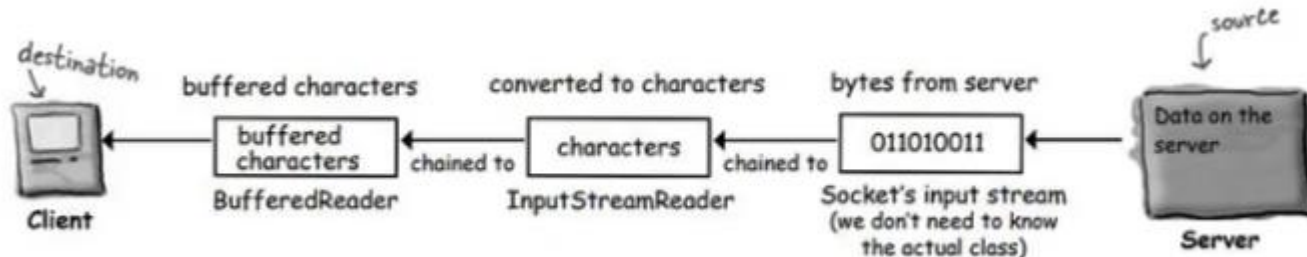
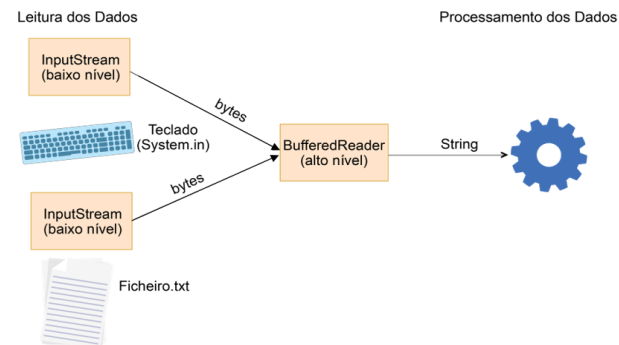
Universidade Católica de Angola

Faculdade de Engenharia Informática

Sistemas Distribuídos e Paralelos I

3.2.1 Classes em java.io

Geralmente usa-se uma classe de baixo-nível para gerir a transferência de dados entre memória e a fonte dos dados (ex: ficheiro), e outra classe de alto-nível para gerir a transferência dos dados da aplicação de e para o stream.





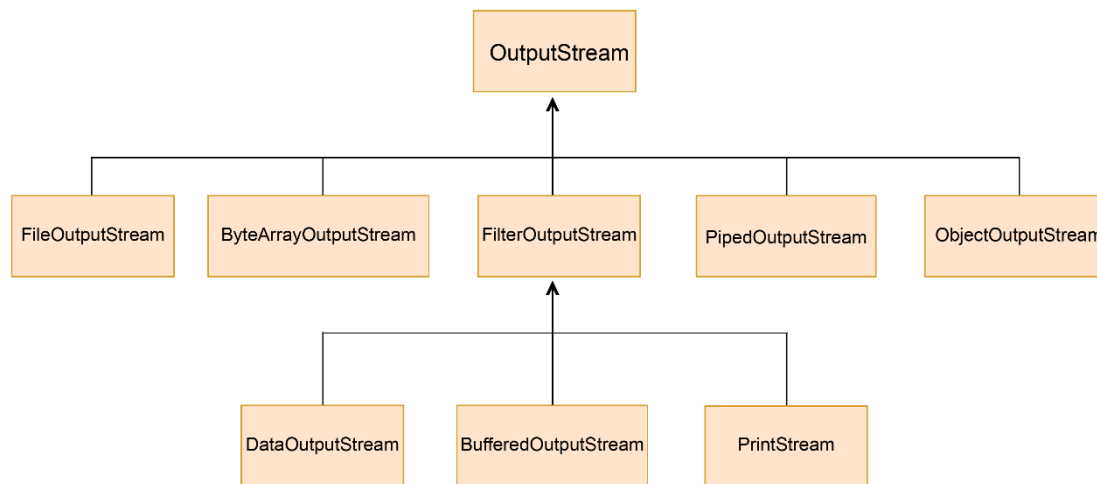
Universidade Católica de Angola

Faculdade de Engenharia Informática

Sistemas Distribuídos e Paralelos I

3.2.1 Classes em `java.io`

- `OutputStream` é uma classe abstracta que transfere sequencialmente os *bytes* para algum destino. Disponibiliza métodos como o `write()`, com a função de escrever em forma de *bytes* para o destino onde vai enviar a informação.



NOTA: há ainda as classes abstractas `Reader` e `Writer`, respectivamente iguais a `InputStream` e a `OutputStream`, mas que são voltadas para a manipulação de caracteres



Universidade Católica de Angola

Faculdade de Engenharia Informática

Sistemas Distribuídos e Paralelos I

3.2.2 Classes em java.net

- Em Java, é possível utilizar a classe `InetAddress` para gerenciar endereços IP e *hostnames* e a classe `NetworkInterface` para representar uma placa de rede.





Universidade Católica de Angola

Faculdade de Engenharia Informática

Sistemas Distribuídos e Paralelos I

// Autor: Celso Paim

// Data: Abril/2026

// Objectivo: pegar o IP e o MAC address do NIC activo e mostrar no ecrã

```
import java.net.*;
```

```
public class BasicNetworking {  
    public static void main ( String args[] ) {  
        try {  
            InetAddress localhost = InetAddress.getLocalHost();  
            System.out.println( "Endereco IP: " + localhost.getHostAddress().trim() );  
  
            NetworkInterface ni = NetworkInterface.getByInetAddress( localhost );  
            byte[] macAddress = ni.getHardwareAddress();  
            String[] macAddressHexa = new String[ macAddress.length ];  
            for ( int i = 0; i < macAddress.length; i++ )  
                macAddressHexa[ i ] = String.format( "%02X", macAddress[ i ] );  
            System.out.println( "Endereco MAC: " + String.join( "-", macAddressHexa ) );  
        }  
        catch( Exception ex ) {  
            ex.printStackTrace();  
        }  
    }  
}
```

BasicNetworking.java

InetAddress

NetworkInterface



Universidade Católica de Angola

Faculdade de Engenharia Informática

Sistemas Distribuídos e Paralelos I

// Autor: Celso Paim

// Data: Abril/2026

// Objectivo: fazer a traducao de um hostname no seu IP e o reverso

DNSResolver.java

InetAddress

```
import java.net.InetAddress;
import java.net.UnknownHostException;

public class DNSResolver {
    public static void main( String[] args ) {
        try {
            String hostname = "unitel.co.ao";
            InetAddress address = InetAddress.getByName( hostname );
            System.out.println( "### Resolucao DNS\n\tHostname: " + hostname );
            System.out.println( "\tIP: " + address.getHostAddress() );

            String IP = "8.8.8.8";
            address = InetAddress.getByName( IP );
            System.out.println( "\n\n### Resolucao Reversa\n\tIP: " + IP );
            System.out.println( "\tHostname: " + address.getHostName() );
        }
        catch ( UnknownHostException e ) {
            System.err.println( "Nao foi possivel resolver o host: " + e.getMessage() );
        }
    }
}
```



Universidade Católica de Angola

Faculdade de Engenharia Informática

Sistemas Distribuídos e Paralelos I

3.2.2 Classes em `java.net`

- Em Java Networking, duas classes se destacam para estabelecer comunicação entre dispositivos: as classes `Socket` e `ServerSocket`, ambas baseadas em TCP, que fornecem a infraestrutura para a construção de uma ampla gama de aplicações em rede, desde simples aplicativos de bate-papo até sistemas cliente-servidor mais complexos.
- A classe `Socket` encapsula a funcionalidade de um *socket* (combinação de um endereço IP e um número de porta, permitindo que um programa específico em um computador se comunique com um programa específico em outro computador). Representa o lado do cliente.
- A classe `ServerSocket` é voltada para o lado do servidor. Sua função principal é escutar conexões de entrada e, quando uma é detectada, criar um novo *socket* para lidar com a comunicação (*streams*) com o cliente que está se conectando.



Universidade Católica de Angola

Faculdade de Engenharia Informática

Sistemas Distribuídos e Paralelos I

// Autor: Celso Paim

// Data: Abril/2026

// Objectivo: executar um servidor que ouvira conexoes TCP e mostrar as msgs recebidas

```
import java.io.*;
```

```
import java.net.*;
```

```
public class Server {  
    public static void main( String[] args ) {  
        try {  
            int port = 12345;  
            ServerSocket serverSocket = new ServerSocket( port );  
            System.out.println( "Servidor iniciado, aguardando por conexoes..." );  
            Socket clientSocket = serverSocket.accept(); // esperando conexoes de clientes  
            BufferedReader input = new BufferedReader( new InputStreamReader( clientSocket.getInputStream() ) );  
            PrintWriter output = new PrintWriter( clientSocket.getOutputStream(), true );  
            String msg = input.readLine();  
            System.out.println( "Mensagem recebida a partir do cliente: " + msg );  
            output.println( "Servidor confirma que recebeu a mensagem: " + msg );  
        }  
        catch ( Exception e ) {  
            e.printStackTrace();  
        }  
    }  
}
```

Server.java

ServerSocket



Universidade Católica de Angola

Faculdade de Engenharia Informática

Sistemas Distribuídos e Paralelos I

// Autor: Celso Paim

// Data: Abril/2026

// Objectivo: executar um cliente que se conectara a um servidor e enviar uma msg

```
import java.io.*;  
import java.net.*;
```

```
public class Client {  
    public static void main( String[] args ) {  
        try {  
            String server_IP = "localhost";  
            int server_port = 12345;  
            Socket socket = new Socket( server_IP, server_port );  
            BufferedReader input = new BufferedReader( new InputStreamReader( socket.getInputStream() ) );  
            PrintWriter output = new PrintWriter( socket.getOutputStream(), true );  
            output.println( "Ola Server!" );  
            System.out.println( input.readLine() );  
        }  
        catch ( Exception e ) {  
            e.printStackTrace();  
        }  
    }  
}
```

Client.java

Socket



Universidade Católica de Angola

Faculdade de Engenharia Informática

Sistemas Distribuídos e Paralelos I

3.2.2 Classes em `java.net`

- `DatagramSocket` representa um *socket* para envio e recebimento de pacotes de datagramas (via UDP). Cada pacote enviado ou recebido em um *socket* de datagrama é endereçado e encaminhados individualmente (vários pacotes enviados de uma máquina para outra podem ser roteados de forma diferente e podem chegar em qualquer ordem, não havendo garantia de entrega).



Universidade Católica de Angola

Faculdade de Engenharia Informática

Sistemas Distribuídos e Paralelos I

// Autor: Celso Paim

// Data: Abril/2026

// Objectivo: executar um servidor que ouvida conexoes UDP e mostrar as msgs recebidas

```
import java.net.*;
```

```
public class ServerUDP {  
    public static void main( String[] args ) {  
        try {  
            int port = 12345;  
            DatagramSocket socket = new DatagramSocket( port );  
            byte[] buffer = new byte[ 1024 ];  
            DatagramPacket packet = new DatagramPacket( buffer, buffer.length );  
            socket.receive( packet );  
            String msg = new String( packet.getData(), 0, packet.getLength() );  
            System.out.println( "Recebida a mensagem: " + msg );  
        }  
        catch ( Exception e ) {  
            e.printStackTrace();  
        }  
    }  
}
```

ServerUDP.java

DatagramSocket



Universidade Católica de Angola

Faculdade de Engenharia Informática

Sistemas Distribuídos e Paralelos I

// Autor: Celso Paim

// Data: Abril/2026

// Objectivo: executar um cliente que se conectara a um servidor com UDP e enviar uma msg

```
import java.net.*;
```

```
public class ClientUDP {  
    public static void main( String[ ] args ) {  
        try {  
            DatagramSocket socket = new DatagramSocket();  
            String msg = "Ola server UDP !";  
            byte[ ] buffer = msg.getBytes();  
            InetAddress server_IP = InetAddress.getByName( "localhost" );  
            int server_port = 12345;  
            DatagramPacket packet = new DatagramPacket( buffer, buffer.length, server_IP, server_port );  
            socket.send( packet );  
            System.out.println( "Mensagem enviada ao servidor" );  
        }  
        catch ( Exception e ) {  
            e.printStackTrace();  
        }  
    }  
}
```

ClientUDP.java

DatagramSocket



Universidade Católica de Angola

Faculdade de Engenharia Informática

Sistemas Distribuídos e Paralelos I

3.2.2 Classes em java.net

- O Java fornece classes dedicadas para lidar com URLs e URIs, facilitando a interação dos programadores com recursos da web.
- Uma URL (*Uniform Resource Locator*) é um tipo específico de URI. Ela não apenas identifica um recurso da web, mas também fornece um meio de localizá-lo usando protocolos específicos, como HTTP, HTTPS, FTP e outros.
- Com a classe `URL`, os programadores podem abrir conexões para recuperar (buscar e ler dados diretamente) ou enviar dados.
- Uma URI (*Uniform Resource Identifier*) é uma sequência de caracteres que identifica um nome ou recurso na Internet. A principal distinção é que, embora todas as URLs sejam URIs, nem todas as URIs são URLs. Isso ocorre porque as URIs englobam URLs e URNs (*Uniform Resource Names*).



Universidade Católica de Angola

Faculdade de Engenharia Informática

Sistemas Distribuídos e Paralelos I

// Autor: Celso Paim

// Data: Abril/2026

// Objectivo: pegar os dados existentes numa determinada URL (pegar o código HTML)

```
import java.net.*;
```

```
import java.io.*;
```

```
public class URLEDemo {
```

```
    public static void main( String[] args ) {
```

```
        try {
```

```
            URL url = new URL( "https://www.cnn.com" );
```

```
            BufferedReader reader = new BufferedReader( new InputStreamReader( url.openStream() ) );
```

```
            String line;
```

```
            while ( ( line = reader.readLine() ) != null ) {
```

```
                System.out.println( line );
```

```
            }
```

```
        }
```

```
        catch ( IOException e ) {
```

```
            System.out.println( "Ocorreu um erro ao consultar a URL: " + e.getMessage() );
```

```
        }
```

```
    }
```

```
}
```

URLEDemo.java

URL



Universidade Católica de Angola

Faculdade de Engenharia Informática

Sistemas Distribuídos e Paralelos I

3.3 Caso de estudo

- Analisar a aplicação de Messenger (**online chat**) construída por Deitel & Deitel que faz recursos a *threads* e a *networking*, e a posterior altera-la para permitir o envio e recepção, não só de mensagens de texto, mas outros tipos de ficheiros como imagens (fotos, vídeos) e audios com até certo tamanho.

Aplicação do lado *Server*:

- `DeitelMessengerServer` (servidor)
- `SocketMessengerConstants` (define as constantes partilhadas como para os portos TCP)
- `MessageListener` (métodos para receber novas mensagens do *chat*)
- `ReceivingThread` (*thread* que ouve as mensagens dos clientes)
- `MulticastSendingThread` (*thread* que envia mensagens aos clientes)

Aplicação do lado *Client*:

- `MessageManager` e `SocketMessageManager` (gerir comunicação com o server)
- `PacketReceivingThread` (*thread* para ouvir as mensagens do servidor)
- `SendingThread` (*thread* para enviar mensagens ao servidor)
- `ClientGUI` (janela com a GUI do cliente)
- `DeitelMessenger` (a aplicação de *chat* do cliente que agrega as várias classes)



Universidade Católica de Angola

Faculdade de Engenharia Informática

Sistemas Distribuídos e Paralelos I

3.3 Caso de estudo

